Rule Based Test Case Reduction Technique using Decision Table

Avinash Gupta[1], Namita Mishra[1], Dharmender Singh Kushwaha[1]
[1]MNNIT Allahabad, Allahabad, India
[1]avinashg.mnnit@gmail.com, [1]dsk@mnnit.ac.in

***Abstract*—Researchers have investigated the use of test suite's reduction techniques to reduce the cost of regression testing. Test suite minimization techniques attempt to remove those test cases from the test suite that have become redundant over time, since the requirements covered by them are also covered by other test cases in the test suite. This paper proposes an approach for software testing, using the concept of decision tables generated from software requirement specification defined by the user. The proposed approach allows testers to make an early estimation of errors and thus, reduce the overall testing cost and time. Moreover, for this type of method since the testing is done using the requirement specification it does not require the tester to have the knowledge of coding or programming logic. The test case redundancy reduction observed for various problems is upto 33%, thus saving significant cost and time.**

***Index Terms*— Decision Table, Open Rule, Testing, Requirement.**

I. Introduction

Software testing is the process of executing a program with the intent of finding errors [9]. It is a very complex, yet important activity and a means to validate and verify software and ensure its quality. Testing techniques are broadly classified into two categories: White-box testing, also known as structural testing, where the implementation of the program or logic of the program will be considered. Black box testing, also known as functional testing, where the structure of the program is not considered. In this, the test cases are decided based on the requirements specifications. In our approach, the functional testing has been chosen to test the software. Black box testing technique is classified as: Boundary Value Analysis (BVA), Robustness, Worst case, Equivalence Partitioning and Decision Table.

The decision tables are human readable rules used to express the test experts or design experts knowledge in a compact form [18]. Decision tables are mainly useful for specifying, analyzing, and testing complex logic of the requirements. These are easily readable and editable by non-technical users (such as business analysts, designers etc.). Decision table logic can be captured and represented using the spread sheets, in a user friendly way and are a good aid for validation and verification process. These are powerful means to find faults both in implementation and specifications.

The different types of testing levels are unit testing, integration testing, system testing, user acceptance testing. In this approach, unit level of testing is selected. In unit testing, each module is tested as individually.

The entire testing process has been divided into four major phases:

- test case generation,
- test case selection,
- test case prioritization followed by
- test case execution.

Decision table based testing is a functional testing technique where the test data are derived from the specified functional requirements without regard to the final program structure [21]. Decision table based testing is considered for the unit testing purpose, as unit testing involves testing of the basic unit of software, i.e. the smallest testable piece of the software. It takes 36 percent to 42 percent of the total resources during various stages of software testing [2]. Thus, it is smarter to test the software in early phases like unit testing phase to minimize the overall testing cost.

This paper proposes a generic framework for test suite generation based on decision table, considering the fact, that the functional requirements of any project or application can be specified by decision tables. This method will help us determine major defects earlier or the errors that might occur in future if not taken care of in prior, and warn the developer to incorporate necessary checks before hand. Moreover, the approach also aims at reducing the redundant test cases that are common in most of the test case generation methods.

The rest of the paper is organized as follows. Section 2 reviews related literature, Section 3 presents the proposed approach and implementation, Section 4 describes performance analysis and comparison results and we conclude the paper and discuss future work in Section 5.

II. Related Research Work

Andrews et al. [5] states that the generation of adequate test cases is difficult and expensive, especially for testing software systems whose input is structurally complex. Zhou et al. [26] states that, research in software testing and analysis has become increasingly active, there is also a growing trend towards combining formal methods and informal techniques for program verification. Apart from the traditional methods of partition testing and cause effect graphing, other methods like random testing [12], path oriented test data generation [22], constraint-based data generation [19], goal oriented test generation [20] and specification based testing [6] have been proposed. Test case generation based on UML activity diagrams, UML state chart diagram and data mutation methods [13] have also been used for modeling the test automation process. Most of these techniques are structural testing

978-1-4799-2572-8/14/$31.00 ©2014 IEEE

techniques that require the understanding of the internal working of the program. Lapierre et al. [24] suggests an approach for automatic unit test data generation for branch coverage using mixed-integer linear programming, execution trees, and symbolic execution. This approach can be useful for both general testing and regression testing after software maintenance and re-engineering activities. Ahmed et al [14] proposes that the goal of software testing is to generate a set of minimal number of test cases such that it reveals as many defects as possible by satisfying particular criteria, called test adequacy criteria, e.g. path coverage. Gorthi et al. [21] says re-executing entire system test suite is an expensive and time consuming activity. To reduce such costs, execution of smaller regression test suite to validate the changed software is suggested. Several techniques, both code-based and model-based that recommend smaller regression test suites have been proposed in the literature. Engstrom et al. [7, 8] conducted a systematic review of empirical evaluations of regression test selection techniques was conducted. No technique was found clearly superior since the results depend on many varying factors. Srivastava et al. [18] used Cause-Effect Graphing (CEG) to identify test cases from a given specification to validate its corresponding implementation. CEG technique can be used to test that software fulfils requirement specification or not. CEG can be converted into decision table. It overcomes existing algorithm's shortcomings and generates all possible test cases.
Sharma and Kushwaha [25] propose testing based on the requirement specification document. Sharma and Chandra [16] proposed generic framework for automating test suite generation based on decision tables, which is a black-box testing technique [15].

III. Proposed Approach and Implementation

The proposed approach aims at designing the test cases after the requirements and definition phase of the SDLC model [10]. It takes the natural language requirements as an input and extracts the necessary conditions/actions from these requirements. The conditions/actions are used to design the decision tables which are further used to determine the rules and generate test cases. Then the test data is determined and the generated test cases are applied on the code for unit testing which is generated in the implementation phase and the test output is generated. The proposed work is represented through the diagram shown in Fig.1:

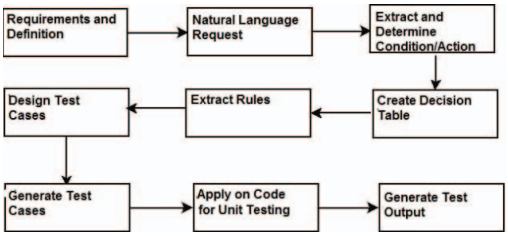

Figure 1. Idea of the Proposed Work

A. Proposed Framework

The proposed framework has been represented through the framework diagram as shown in Fig.2. The entire framework of the proposed work has been divided into 3 major steps which are further divided into sub steps as explained below:

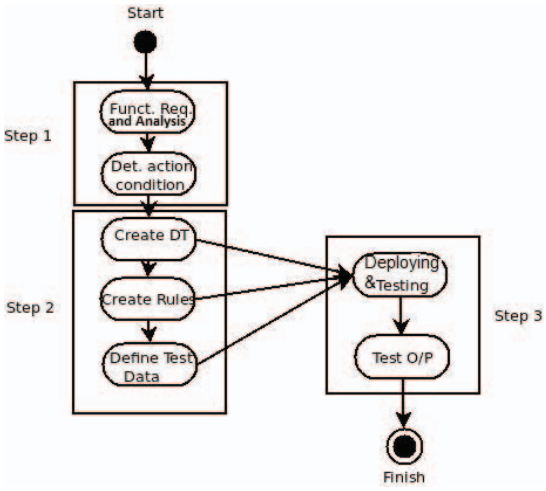

Figure 2. Framework for the Proposed Approach

Step 1: Functional Requirements Analysis and Condition/Action Determination:

This step has been divided into 2 sub steps that are:
Step 1.1: Functional Requirements Analysis: This step takes functional requirements as its input and identifies the major steps or decisions which need to be made to achieve the goal presented in the requirements specification.
Step 1.2: Condition/Action Determination: In this step, after the analysis of the requirements, we determine all possible conditions and their corresponding actions that are specific to the decisions identified in the previous step.

Step 2: Input Generation for Rule Deployment and Testing:

This step has been divided into 3 sub steps that are:
Step 2.1: Create Decision Table: We map the conditions and actions into spreadsheet to generate the decision tables which tells us what action needs to be performed when a certain condition is met. Now, the generated decision tables are repeatedly analysed to remove the redundant conditions that are present. Removing redundancy leads to minimization of the decision table. Minimum conditions or cases in the decision table means minimum number of test cases required to test our application. The decision tables are represented in the format acceptable by the OpenRules tool [17] used in the step 3 i.e. in the spreadsheet format.
Step 2.2: Create Corresponding Rules: Using the decision table formed in the previous step, we create the rules by identifying the various objects related to the requirements specification and the relationship between

these objects. The rules are presented in the spreadsheet in the form acceptable by the OpenRules tool [17] used in the step 3.

Step 2.3: Define Test Data: Using decision tables generated in the previous step, we determine what will be the test data applicable to user scenario and the expected output. The test data, data and data types are defined in the spreadsheet that is acceptable by the OpenRules tool [17] used in the step 3.

Step 3: Rules Deployment and Testing Using OpenRules: This step has been divided into 2 sub steps that are:

Step 3.1: Rules Deployment and Testing using OpenRules: In this step, all the results of the previous three steps viz. decision tables, rules and the test data are passed to the OpenRules tool [17] for their deployment. Now, the tool will deploy these decision tables and rules and use the defined test data to verify the decision tables. The verification is done by checking whether the desired/expected output is obtained or not.

Step 3.2: Generation of Test Output: The test output is generated after the rules deployment and testing using the test data.

B. Implementation

For the proposed work, we have been customised OpenRules Tool Architecture [17] according to the requirement. The components that were not required were removed from the original architecture and the modified version of the tool is taken into use. The customised tool architecture is shown in Fig.3:

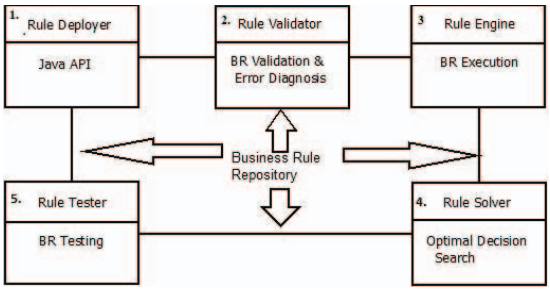

Figure 3. Customized OpenRules Tool Architecture

The entire working architecture of the tool has been divided into 5 parts:

1. Rule Deployer: These rules can be deployed in four ways as described below:

- As components of Java Applications
- As Web Services
- As presentation-oriented Web Applications and,
- On Cloud.

In the proposed work only the first option i.e. the Java API to deploy the rules is considered.

2. Rule Validator: It is a component that is capable to catch as many as possible errors in rules at design time rather at run-time when it is basically too late. Validation can be done in two ways:

- **Rule Validation with Eclipse Plug-In:** OpenRules Eclipse Plug-in diagnoses any errors in Excel-files before you deploy or start your OpenRules-based application. It automatically diagnoses errors in the Excel-files and displays the appropriate error message(s) inside Eclipse views.
- **Batch Rule Validator:** If using a stand-alone version of OpenRules without the Eclipse IDE, Excel files can still be validated. OpenRules provides a special validation module, the batch Rule Validator, that allows to test all of the xls-files contained in rule project.

3. Rule Engine: It is used to execute different rule sets and methods specified in Excel files using application-specific business objects. OpenRulesEngine can be invoked from any Java application using a simple Java API. The working of Rule Engine is explained as under:

- **Rules Execution Logic:** When OpenRulesEngine is created it downloads the main xls-file and all other files using the "include" properties in the Environment tables. The engine executes the rules starting with the main method defined for this particular engine run.
- **Relationships between Rules inside Decision Tables:** The execution logic inside decision tables is very intuitive. It does not assume any implicit execution logic and executes rules in the order specified by a rule designer. All rules are executed one-by-one in the order they are placed in the rules table. There is a simple rule that governs rules execution inside a rules table.

For standard vertical decision tables, all rules (rows) are executed in top-to-bottom order. For horizontal decision tables, all rules (columns) are executed in left-to-right order. The execution logic of one rule (one row in the decision table) is as follows:

- IF ALL conditions are satisfied THEN execute ALL actions.
- If at least one condition is violated (the proper code evaluation produces false), all other conditions in the same rule (row) are ignored and are not evaluated.
- The absence of a parameter in a condition cell means the condition is always true.
- Actions are executed only if ALL conditions in the same row evaluated to be true. Action cells with no parameters are ignored.

4. Rule Solver: It is used to model and solve scheduling, resource allocation, configuration, and other constraint satisfaction and optimization problems.

5. Rule Tester: It provides the ability to define new data/object types and create objects of these types directly in Excel spreadsheets. To organize a rules testing environment, the rule designer usually does the following:

- Specifies his/her own data types as Excel "Datatype" tables

- Creates test instances of objects of these types using Excel "Data" tables and,
- Creates an Excel Method table that calls the proper appropriate Rules tables and passes to them the instances of test data.

This approach allows business analysts to define business terms and facts in Excel without worrying about their actual implementation in Java or XML.

C. *Illustrative Example*

The proposed scheme is illustrated by the example.
DetermineCustomerGreeting (Requirements Specification): This project demonstrates how to develop a simple decision model that should decide how to greet a customer during different times of the day.
This requirements specification is taken as an input to the proposed framework. Above requirements specification is processed using the proposed framework.

Step 1: Functional Requirements Analysis and Condition/Action Determination:

Step 1.1: Functional Requirements Analysis:
Input: This project demonstrates how to develop a simple decision model that should decide how to greet a customer during different times of the day.
Output: After the analysis of the requirements specification following were identified:

- The application will take current time from the system.
- According to the time range the greeting message must vary e.g. if time range is between 0 to 11 hour then the greeting message must be Good Morning and if it is between 12 to 17 hour then the greeting message must be Good Afternoon.
- According to the gender and marital status the salutation of the customer must vary e.g. if a customer is female and married then the salutation must be Mrs. NameOfTheCustomer and, if the customer is a male then regardless of whether he is married or not the salutation must be Mr. NameOfTheCustomer.

Step 1.2: Condition/Action Determination:
Here we identify all possible conditions and their corresponding actions. e.g.
Condition: Current hour is between 17 to 22 hours customer name is Jiya and is a married male.
Action: Set salutation as Mrs. Jiya and Greeting as Good Morning and print message as Good Evening Mrs.Jiya.

Step 2: Input Generation for Rule Deployment and Testing:

Step 2.1: Create Decision Table: Now, on the basis of the results of the step 1, decision tables are determined. In this case, the decision tables are shown:

Decision DetermineCustomerGreeting	
Decisions	**Execute Decision Tables**
	:= System.out.println(customers[0])
Define Current Time	:= DefineCurrentHour()
Define Greeting Word	:= DefineGreeting()
Define Salutation Word	:= DefineSalutation()
Define Customer Greeting	:= decision().put("result", response.greeting + ", " + response.salutation + " " + customers[0].name + "!")

Figure 4. Snapshot of Decision Table: DetermineCustomerGreeting

Fig.4 is the main decision table that will be used to call other decision tables that are used to define the rules to execute the decision logic. It is accompanied by other include files defined in the Env.xls that tell what all files need to be included to execute the logic.

Environment	
	../include/Rules.xls
	../include/Data.xls
	../include/Glossary.xls
Include	../../../openrules.config/DecisionTemplates.xls

Figure 5. Snapshot of Environment

In Fig.5, Env.xls tells that the execution will need or will include Rules.xls, Data.xls, Glossary.xls and, DecisionTemplates.xls that are a part of the openrules.config.
Step 2.2: Create Corresponding Rules: This will include all rule files that will define the rules. Like in this case, on the execution of the xls file Decision.xls the first rule that is called is DefineCurrentHour that is defined in the xls file named Rules.xls. The CurrentHour method is shown in Fig.6:

```
Method void DefineCurrentHour()
int  hour =
Calendar.getInstance().get(Calendar.HOUR_OF_DAY);
setInt("Current Hour",hour);
System.out.println("Current Hour = " + hour);
```

Figure 6. Snapshot of Decision Table CurrentHour

The next rule that is called is DefineGreeting. The snapshot is shown in Fig.7:

DecisionTableDefineGreeting					
Condition		Condition		Conclusion	
Current		**Current**		**Greeting**	
>=	0	<=	11	Is	Good Morning
>	11	<=	17	Is	Good Afternoon
>	17	<=	22	Is	Good Evening
>	22	<=	24	Is	Good Night

Figure 7. Snapshot of Decision Table: DefineGreeting

The next lined up rule is DefineSalutation. The snapshot is shown in Fig.8:

DecisionTableDefineSalutation					
Condition		Condition		Conclusion	
Gender		**Marital Status**		**Salutation**	
Is	Male			Is	Mr.
Is	Female	Is	Married	Is	Mrs.
Is	Female	Is	Single	Is	Ms.

Figure 8. Snapshot of Decision Table: DefineSalutation

Step 2.3: Define Test Data: This file Data.xls will contain all the data types of the business objects and the test data that will be used to test the rules that are defined. The data type for response object is shown in Fig.9:

Datatype Response	
String	Greeting
String	Salutation
Int	Hour

Figure 9. Snapshot of Data Types for Response Objects

The data type for customer object is shown in Fig.10:

Datatype Customer	
String	Name
String	maritalStatus
String	Gender
Date	Dob

Figure 10. Snapshot of Datatype for Customer

The sample test data is shown in Fig.11:

Data Customer customers			
name	maritalStatus	gender	Dob
Customer Name	Marital Status	Gender	Date of Birth
Jiya	Married	Female	24-04-1985
Jhanavi	Single	Female	10-02-1983
Sumit	Single	Male	19-10-1980

Figure 11. Snapshot of Test Data

The response objects are shown in Fig.12:

Variable Response response		
greeting	salutation	hour
Greeting	Salutation	Current Hour
?	?	8

Figure 12. Snapshot of Response Objects

Note, these files will be called and executed by calling a file called Main.java.

Main.java

```
package hello;
/* A simple Java launcher for Rules.xls with data defined
in Excel file Data.xls */
import com.openrules.ruleengine.Decision;
public class Main {
public static void main(String[] args) {
String fileName = "file:rules/main/Decision.xls";
Decision decision = new
Decision("DetermineCustomerGreeting",fileName);
decision.execute();
System.out.println("Decision: " + decision.get("result"));
} }
```

Step 3: Rules Deployment and Testing Using OpenRules:

Step 3.1: Rules Deployment and Testing using OpenRules: All the files generated in the step 2 i.e. Decision.xls, Rules.xls and Data.xls are fed as input to the OpenRules tool for deployment and testing of the rules according to the test data.

Step 3.2: Generation of Test Output: The test output is generated after the OpenRules has deployed and tested the decision table and rules according to the test data defined by the Data.xls. The test output is shown in Fig.13.

```
*** Decision DetermineCustomerGreeting ***
Decision has been initialized
Decision Run has been initialized
Customer(id=0) {
name=Jiya
dob=Wed Apr 24 22:44:19 IST 1985
gender=Female
maritalStatus=Married
}
Decision DetermineCustomerGreeting: Define Current Time
Current Hour = 22
Decision DetermineCustomerGreeting: Define Greeting Word
Conclusion: Greeting Is Good Evening
Decision DetermineCustomerGreeting: Define Salutation Word
Conclusion: Salutation Is Mrs.
Decision DetermineCustomerGreeting: Define Customer
Greeting
Decision has been finalized
Decision: Good Evening, Mrs.Jiya!
```

Figure 13. Snapshot of Test Output

According to the test output it can be inferred that according to the current time and the gender and marital status of the customer, the application gives the expected output. Thus, this is validating the rules. Hence, it is evident that the application can be tested using the rules or the decision tables that are created from the requirements specification without being bothered of the implementation code. Thus, this will help testers to make an early estimation of the errors that might come, validate requirements.

IV. Performance Analysis

For the performance analysis, the approach is compared with the code based approach that generates decision tables from code and consequently the test cases. Further the results of comparison of the two approaches are mapped into comparison table and comparison graph is drawn for the analysis. The detail procedure is explained in the following section.

A. Comparison

For the comparison, taking the same requirement specification the decision tables were created using the proposed approach and the redundant cases that were present in case of code based approach were removed to give the minimal set of test cases that are possible.

Example: The proposed scheme is illustrated by the example DetermineCustomerGreeting below.

DetermineCustomerGreeting Requirement Specification: This project demonstrates how to develop a simple decision model that should decide how to greet a customer during different times of the day.

According to the code based approach: The code to implement the above stated requirement specification will have the control structure as shown Fig.14:

```
Public class Greeting{
Public voidDetermine Greeting(intcurrentHour, String maritalStatus, String
gender,String name)
{
if(gender.equals"Male"&&maritalSatus.equals="Married")
salutation="Mr.;
elseif(gender.equals"Male"&&maritalSatus.equals="Single")
salutation="Mr.;
elseif(gender.equals"Female"&&maritalStatus.equals="Married")
salutation="Mrs";
elseif(gender.equals"Female"&&maritalStatus.equals="Single")
salutation="Ms";

if((currentHour>=0 &&currentHour<=11)
greeting="GoodMorning";
elseif(currentHour> 11 &&currentHour<=17)
greeting="GoodAfternoon";
elseif(currentHour>17 &&currentHour<=22)
greeting="Good Evening"
elseif(currentHour>22&&currentHour<=24)
 greeting="Good Night";
System.out.println"greeting.+salutation+name");
}
}
```

Figure 14. Control Structure for the Requirement Specification

According to the code, the corresponding test cases will be as shown in the Table: 1.

TABLE I. TEST CASES FOR CODE BASED APPROACH

Sr. No	Gender	Marital Status	Current Hour Range	Salutation	Greeting
1	Male	Married	>=0 hour<=11	Mr.	Good Morning
2	Male	Married	> 11 hour<=17	Mr.	Good Afternoon
3	Male	Married	>17 hour<=22	Mr.	Good Evening
4	Male	Married	>22 hour<=24	Mr.	Good Night
5	Male	Single	>=0 hour<=11	Mr.	Good Morning
6	Male	Single	>11 hour<=17	Mr.	Good Afternoon
7	Male	Single	>17 hour<=22	Mr.	Good Evening
8	Male	Single	>22 hour<=24	Mr.	Good Night
9	Female	Married	>=0 hour<=11	Mrs.	Good Morning
10	Female	Married	> 11 hour<=17	Mrs.	Good Afternoon
11	Female	Married	>17 hour<=22	Mrs.	Good Evening
12	Female	Married	>22 hour<=24	Mrs.	Good Night
13	Female	Single	>=0 hour<=11	Ms.	Good Morning
14	Female	Single	> 11 hour<=17	Ms.	Good Afternoon
15	Female	Single	>17 hour<=22	Ms.	Good Evening
16	Female	Single	>22 hour<=24	Ms.	Good Night

According to the proposed approach: The decision tables that will be created for the stated requirement specification are as shown in Fig.15 and Fig.16

DecisionTableDefineSalutation					
Condition		Condition		Conclusion	
Gender		Marital Status		Salutation	
Is	Male			Is	Mr.
Is	Female	Is	Married	Is	Mrs.
Is	Female	Is	Single	Is	Ms.

Figure 15. Snapshot of Decision Table: DefineSalutation

DecisionTableDefineGreeting					
Condition		Condition		Conclusion	
Current		Current		Greeting	
>=	0	<=	11	Is	Good Morning
>	11	<=	17	Is	Good Afternoon
>	17	<=	22	Is	Good Evening
>	22	<=	24	Is	Good Night

Figure 16. Snapshot of Decision Table: DefineGreeting

Taking into consideration the decision tables created, the test cases through the proposed approach are shown in Table: 2:

TABLE II. TEST CASES FOR CODE PROPOSED APPROACH

Sr. No.	Gender	Marital Status	Current Hour Range	Salutation	Greeting
1	Male	Married	>=0 hour<=11	Mr.	Good Morning
2	Male	Married	> 11 hour<=17	Mr.	Good Afternoon
3	Male	Married	>17 hour<=22	Mr.	Good Evening
4	Male	Married	>22 hour<=24	Mr.	Good Night
5	Female	Married	>=0 hour<=11	Mrs.	Good Morning
6	Female	Married	> 11 hour<=17	Mrs.	Good Afternoon
7	Female	Married	>17 hour<=22	Mrs.	Good Evening
8	Female	Married	>22 hour<=24	Mrs.	Good Night
9	Female	Single	>=0 hour<=11	Ms.	Good Morning
10	Female	Single	> 11	Ms.	Good

			hour<=17		Afternoon
11	Female	Single	>17 hour<=22	Ms.	Good Evening
12	Female	Single	>22 hour<=24	Ms.	Good Night

It is observed that test cases (5,6,7,8) are redundant in Table:1 because they were checking for the salutation word based on the marital status value as single, which is not required because, independent of the fact whether the male is married or single the salutation word will always be Mr.. Therefore, it is not needed for an extra check there.

This fact is well considered in the decision table based approach and thus, these redundant test cases (5,6,7,8) have been removed from Table:1 as shown in Table 2. Thereby, reducing the total number of test cases.

Percentage Redundancy Reduction (Rr):

Rr=((Nc-Np)/Np)*100

where,

Nc=Number of test cases in code based approach and,

Np=Number of test cases in proposed approach.\\

Calculation of Rr:

Total number of test cases through code based approach=Nc=16.

Total number of test cases through proposed approach=Np=12.

Using the above formula:

Rr =((16-12)/16)*100=33(in percent).

The summarized view of the sample problem statements and their source along with the comparative results is shown in the Table: 3.

TABLE III. COMPARISON TABLE

S. No.	Problem Statement	Problem Source	Nc	Np	Rr (%)
1	Hello Customer	Self Made	16	12	33
2	Customer Senior	Self Made	2	2	0
3	Leap Year	Self Made	6	6	0
4	Simple Calculator	Self Made	8	8	0
5	Vacation Days	Self Made	8	6	33
6	Auto Insurance premium Calculator	Self Made	34	30	13.3
7	Patient Therapy	http://openrules.com/pdf/	14	10	40
8	Loan Pre Qualification	www.kpiusa.com	28	22	27.2
9	Business customer Module	http://openrules.com	38	34	11.7
10	Income tax calculator	http://openrules.com	27	22	22.7

The comparison graph, showing the comparison between the number of test cases generated from our approach and the code based approach is shown in Fig.17:

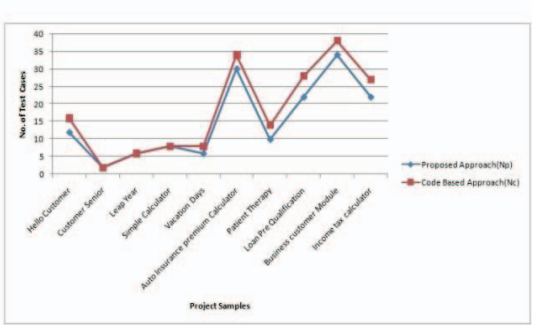

Figure 17. Comparative Results of the Two Approaches: Comparison Graph

B. Comparison Results

From the Rr column in the Table 3, it can be seen that the approach is able to reduce the redundancy in the test cases in many of the sample problem statements taken for the analysis. Moreover, in the graph it can be seen that the line that is showing the results of the proposed approach is never going above the one that is showing the results of the code based approach. Therefore, it is evident that the approach is successful in reducing the number of test cases.

CONCLUSIONS

This paper proposes an approach to generate the test case in such a way that the tester could do the early error estimation, and thus, reduce the overall testing cost and time. This approach reduces the redundancy and generate minimal test suite, i.e. the test suite with minimum number of test cases. Our method proposes the testing using the requirement specification and it does not require the tester to have the knowledge of coding or programming logic.

The test case redundancy reduction observed for various problems is upto 33%, thus saving significant cost and time.

REFERENCES

[1] A. Jamoussi. An automated tool for efficiently generating a massive number of random test cases. In *High-Assurance Systems Engineering Workshop, 1997., Proceedings*, pages 104–107. IEEE, 1997.

[2] B. Beizer. *Software testing techniques*. Dreamtech Press, 2002.

[3] B. Korel. Automated software test data generation. *Software Engineering, IEEE Transactions on*, 16(8):870–879, 1990.

[4] C. Kaner, J. Falk, and H. Q. Nguyen, *Testing Computer Software*, 2nd ed. New York: Van Nostrand Reinhold, 1993.

[5] D. M. Andrews and J. P. Benson. An automated program testing methodology and its implementation. In *Proceedings of the 5th international conference on Software engineering,* pages 254–261. IEEE Press, 1981.

[6] D. Marinov and S. Khurshid. Testera: A novel framework for automated testing of java programs. In *Automated Software Engineering, 2001.(ASE 2001). Proceedings. 16th Annual International Conference on*, pages 22–31. IEEE, 2001.

[7] E. Engstr¨om and P. Runeson. A qualitative survey of regression testing practices. In *Product-Focused Software Process Improvement*, pages 3–16. Springer, 2010.

[8] E. Engstr¨om, P. Runeson, and M. Skoglund. A systematic review on regression test selection techniques. *Information and Software Technology*, 52(1):14–30, 2010.

[9] G. J. Myers, C. Sandler, and T. Badgett. *The art of software testing*. Wiley, 2011.

[10] I. Sommerville. Software process models. *ACM Computing Surveys (CSUR)*, 28(1):269–271, 1996.

[11] "J Unit web site," 2005, www.junit.org.

[12] K. Claessen and J. Hughes. Quickcheck: a lightweight tool for random testing of haskell programs. *Acmsigplan notices*, 46(4):53–64, 2011.

[13] L. Shan and H. Zhu. Generating structurally complex test cases by data mutation: A case study of testing an automated modelling tool. *The Computer Journal*, 52(5):571–588, 2009.

[14] M. A. Ahmed and I. Hermadi. Ga-based multiple paths test data generator. *Comput. Oper. Res.,* 35(10):3107–3124, Oct. 2008.

[15] M. A. Khan and M. Sadiq. Analysis of black box software testing techniques: A case study. In *Current Trends in Information Technology (CTIT), 2011 International Conference and Workshop on*, pages 1–5. IEEE, 2011.

[16] M. Sharma and B. S. Chandra. Automatic generation of test suites from decision table-theory and implementation. In *Software Engineering Advances (ICSEA), 2010 Fifth International Conference on*, pages 459–464. IEEE, 2010.

[17] OpenRules® architecture and related components http://openrules.com/architecture.htm

[18] P. R. Srivastava, P. Patel, and S. Chatrola. Cause effect graph to decision table generation. *ACM SIGSOFT Software Engineering Notes*, 34(2):1–4, 2009.

[19] R. DeMilli and A. J. Offutt. Constraint-based automatic test data generation. *Software Engineering, IEEE Transactions on*, 17(9):900–910, 1991.

[20] R. Ferguson and B. Korel. Generating test data for distributed software using the chaining approach. *Information and Software Technology*, 38(5):343–353, 1996.

[21] R. P. Gorthi, A. Pasala, K. K. Chanduka, and B. Leong. Specification-based approach to select regression test suite to validate changed software. In *Software Engineering Conference, 2008. APSEC'08. 15th Asia-Pacific*, pages 153–160. IEEE, 2008.

[22] S. Beydeda and V. Gruhn. Test data generation based on binary search for class-level testing. In Book of Abstracts*: ACS/IEEE International Conference on Computer Systems and Applications*, page 129, 2003.

[23] S. Eldh, H. Hansson, S. Punnekkat, A. Pettersson, and D. Sundmark. A framework for comparing efficiency, effectiveness and applicability of software testing techniques. In Testing: *Academic and Industrial Conference-Practice And Research Techniques, 2006. TAIC PART 2006. Proceedings*, pages 159–170. IEEE, 2006.

[24] S. Lapierre, E. Merlo, G. Savard, G. Antoniol, R. Fiutem, and P. Tonelia. Automatic unit test data generation using mixed-integer linear programming and execution trees. In *Software Maintenance, 1999.(ICSM'99) Proceedings. IEEE International Conference on*, pages 189–198. IEEE, 1999.

[25] A, Sharma and D.S.Kushwaha, "An Empirical Approach for Early Estimation of Software Testing Effort", In CSI Transactions on ICT, Springer Publisher, Vol 1., Issue 1, 51-66, 2012.

[26] Z. Zhou, B. Scholz, and G. Denaro. Automated software testing and analysis: Techniques, practices and tools. In *System Sciences, 2007. HICSS 2007. 40th Annual Hawaii International Conference on*, pages 260–260. IEEE, 2007.